

ERCOT GINR Feasibility Screening ML Service

End-to-End Architecture, Business Problem, Solution Design & Interview Explanation

Project Name

ERCOT Generation Interconnection Request (GINR) Feasibility Screening Machine Learning Service

Technologies Used

- Python
- Scikit-Learn
- Pandas
- NumPy
- Flask REST API
- Oracle Database
- Spring Boot
- Joblib
- REST Services
- Git
- Jenkins
- Docker (Future Enhancement)

1. Executive Summary

ERCOT receives a large number of Generation Interconnection Requests (GINRs) from renewable and traditional energy providers requesting connection to the Texas power grid.

The feasibility screening process requires engineers to manually validate project information, payment status, connectivity details, point of interconnection information, thermal constraints, and network readiness before approving a request.

The objective of this project was to build a Machine Learning based screening service capable of predicting whether a GINR submission would pass or fail feasibility screening before engineers perform manual reviews.

The solution helps engineers prioritize reviews, reduce operational costs, improve turnaround times, and increase productivity.

2. Business Problem

Existing Process

Before this solution, every submitted GINR was manually reviewed by engineers.

The process involved:

- Verifying payment information
- Reviewing project capacity
- Checking connectivity nodes
- Validating Point of Interconnection (POI)
- Reviewing thermal limits
- Verifying study requirements

This process was time-consuming and resource-intensive.

Challenges

1. High volume of requests
2. Long processing times
3. Manual effort required
4. Engineering resource constraints
5. Delayed approvals
6. High number of incomplete submissions
7. Lack of risk-based prioritization

Business Impact

- Increased backlog
 - Reduced efficiency
 - Delayed customer onboarding
 - Higher operational costs
-

3. Business Objective

The business requested an intelligent system capable of:

Predicting

Whether a submitted GINR is likely to pass feasibility screening.

Providing

- Pass Probability

- Risk Score
- Early Warning Indicators

Benefits

- Faster processing
- Better prioritization
- Reduced manual effort
- Improved engineer productivity
- Faster customer response times

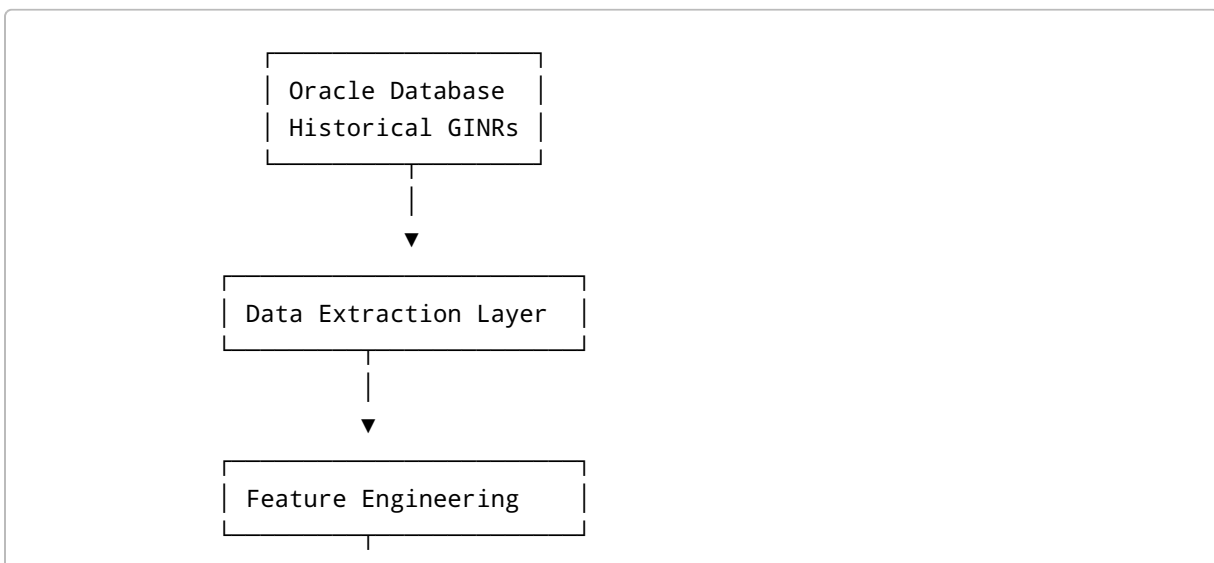
4. Solution Overview

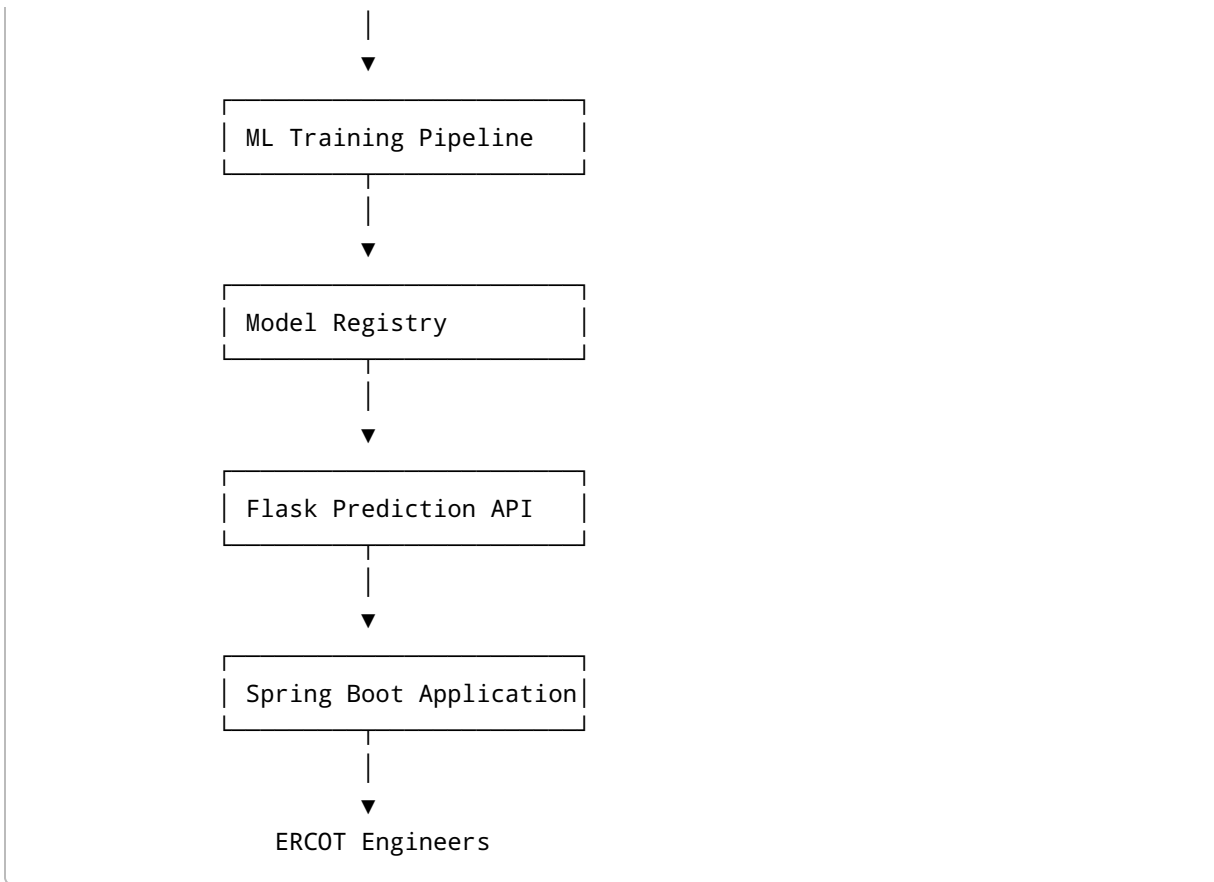
We designed a Machine Learning service that learns from historical GINR submission data and predicts approval probability for new requests.

The solution consists of:

1. Data Extraction Layer
2. Feature Engineering Layer
3. Machine Learning Pipeline
4. Model Registry
5. Prediction API
6. Spring Boot Integration
7. Monitoring and Drift Detection

5. High-Level Architecture





6. Step-by-Step Execution Flow

Step 1 – Data Collection

Historical GINR records were extracted from Oracle databases.

Data Sources:

- GINR Requests
- Payment Information
- POI Data
- Connectivity Nodes
- Study Results
- Engineering Reports

Historical data from multiple years was collected.

Output:

Step 2 – Data Cleaning

Raw data contained:

- Missing values
- Duplicate submissions
- Inconsistent formats
- Invalid records

Activities Performed:

- Remove duplicates
- Handle null values
- Normalize fields
- Standardize formats

Output:

Cleaned Dataset

Step 3 – Feature Engineering

Raw business data was transformed into machine learning features.

Numerical Features

- Project MW
- POI Voltage
- Thermal Margin
- Months To COD

Binary Features

- Payment Completed
- Missing Documents
- Node Available

Categorical Features

- Resource Type
- Lead TDSP
- Interconnection Type

Additional Processing:

- One Hot Encoding
- Scaling
- Missing Value Imputation

Output:

ML Ready Dataset

Step 4 – Model Training

Machine Learning Algorithm:

Logistic Regression

Reason for Selection:

- Explainable
- Fast Training
- Easy Maintenance
- Probability Based Predictions
- Good Performance on Structured Data

Pipeline Components:

Input Data ↓ Imputer ↓ Encoder ↓ Scaler ↓ Logistic Regression ↓ Trained Model

Step 5 – Model Evaluation

Metrics Used:

Accuracy

Overall prediction correctness.

Precision

How many predicted failures were actually failures.

Recall

How many actual failures were correctly identified.

ROC-AUC

Overall model quality.

Business Priority:

Recall

Reason:

Missing a problematic request is more expensive than reviewing an extra request.

Step 6 – Model Versioning

Every trained model is saved with version information.

Examples:

- ginr_model_v1.0.joblib
- ginr_model_v1.1.joblib

Stored Metadata:

- Training Date
- Feature List
- Evaluation Metrics
- Dataset Version

Benefits:

- Rollback Support
 - Auditability
 - Governance
-

Step 7 – Prediction API

The trained model is exposed through a Flask REST API.

Endpoints:

Health Check

GET /api/v1/health

Single Prediction

POST /api/v1/score

Batch Prediction

POST /api/v1/score/batch

Model Information

GET /api/v1/model/info

Drift Monitoring

GET /api/v1/drift

Step 8 – Real-Time Prediction Flow

1. User submits GINR request.
2. Spring Boot application receives request.
3. Spring Boot calls Flask Prediction API.
4. Flask validates incoming payload.
5. Feature Engineering pipeline executes.
6. Trained model performs prediction.
7. Probability score generated.
8. Risk category assigned.
9. Response returned to application.

Example Response:

```
{ "prediction": "PASS", "probability": 0.89, "risk": "LOW" }
```

Step 9 – Risk Categorization

Probability > 80%

LOW RISK

Probability Between 50% and 80%

MEDIUM RISK

Probability Below 50%

HIGH RISK

This enables engineers to prioritize their workload.

Step 10 – Monitoring & Drift Detection

The solution continuously monitors incoming production data.

Monitored Areas:

- Feature Distribution Changes
- New Resource Types
- Data Quality Issues
- Prediction Patterns

If drift exceeds thresholds:

- Alert Generated
 - Retraining Triggered
-

7. Deployment Architecture

Training Environment

Oracle Database ↓ Python Training Pipeline ↓ Model Artifact ↓ Model Registry

Production Environment

RARF UI ↓ Spring Boot ↓ REST API Call ↓ Flask ML Service ↓ Prediction Response

8. Business Benefits Delivered

Before Implementation

- 100% Manual Screening
- Slow Processing
- Large Backlog
- No Prioritization

After Implementation

- Automated Risk Assessment
- Faster Processing
- Reduced Engineering Effort
- Better Prioritization
- Improved Turnaround Time

Expected Benefits

- 40–60% reduction in manual review effort
 - Faster approval cycle
 - Better utilization of engineering resources
 - Improved customer satisfaction
-

9. My Role in the Project

As a Senior Software Engineer / AI Engineer, my responsibilities included:

- Understanding business requirements
 - Analyzing historical GINR data
 - Designing ML architecture
 - Building feature engineering pipelines
 - Training and evaluating ML models
 - Developing Flask prediction services
 - Integrating with Spring Boot applications
 - Implementing monitoring and versioning
 - Supporting deployment and testing
-

10. Interview Explanation (5 Minutes)

ERCOT processes hundreds of Generation Interconnection Requests from renewable and traditional energy providers seeking connection to the Texas grid.

The existing feasibility screening process was manual and required engineers to review payment information, connectivity details, point of interconnection information, and network constraints. This resulted in delays, high operational costs, and limited scalability.

To address this challenge, we built a Machine Learning based Feasibility Screening Service that predicts whether a GINR is likely to pass feasibility review before engineers begin manual analysis.

Historical data was extracted from Oracle databases and transformed into machine learning features such as project size, payment status, POI voltage, connectivity information, and thermal margins.

We developed a Scikit-Learn pipeline using preprocessing components such as SimpleImputer, OneHotEncoder, and StandardScaler, followed by a Logistic Regression model. The model was evaluated using Accuracy, Precision, Recall, and ROC-AUC metrics.

The trained model was deployed as a Flask REST API. Whenever a new GINR is submitted, the Spring Boot application calls the API to obtain pass probability and risk classification.

This enables engineers to focus on high-risk requests first, reduce manual effort, improve turnaround times, and create a scalable feasibility screening process.

Additional capabilities such as model versioning, monitoring, and drift detection ensure long-term reliability and governance.